

# Análisis comparativo de técnicas de caja negra

## Resumen

La suite de SecureBank combina 125 casos automatizados sobre un modelo bancario en memoria. Todos pasan y la ejecución conjunta alcanza 100 % de líneas y ramas instrumentadas. Estos resultados verifican el contrato declarado, pero no demuestran que una plataforma bancaria real sea segura. Para comparar efectividad se ejecutaron cinco mutaciones deliberadas en copias temporales, preservando la implementación original y los registros de cada experimento.

## Efectividad de las técnicas

Las particiones de equivalencia fueron la técnica más sencilla de aplicar. Clasificar importes, tipos de cuenta, saldos, beneficiarios e intervalos permitió cubrir comportamientos distintos sin recorrer todos los valores posibles. Sus 38 casos detectaron dos mutaciones. Su principal debilidad fue que representantes normales, como una transferencia de quinientos dólares, no distinguen correctamente una comparación estricta de una inclusiva. La selección del representante importa tanto como identificar la clase.

Los valores límite aportaron 37 casos y detectaron cuatro mutaciones. Fueron especialmente útiles para el centavo mínimo, los topes diarios y la exención de comisiones. Probar exactamente mil dólares permitió detectar una exención indebidamente inclusiva que las particiones generales no revelaron. El reloj controlado también permitió observar el comportamiento inmediatamente antes y después de medianoche. Su costo principal estuvo en establecer expectativas inequívocas cuando el enunciado usa expresiones ligeramente diferentes para describir el mismo umbral.

Las tablas de decisión ofrecieron la mayor detección en este experimento: cinco de cinco mutaciones, con 38 casos. Explicitar fondos, límite y estado produjo ocho combinaciones de transferencia. Para pagos se conservaron dieciséis combinaciones, señalando cuándo una condición resulta irrelevante. Esta técnica fue la más laboriosa porque exigió preparar estados y saldos sin escribir atributos internos, además de declarar la precedencia de errores. Su ventaja fue revelar interacciones que desaparecen cuando cada condición se prueba aisladamente.

Las transiciones de estado reunieron doce casos y detectaron dos mutaciones. Su aporte distintivo fue comprobar secuencias: suspender, rechazar operaciones y recuperar acceso mediante depósito; congelar y descongelar; cerrar permanentemente. Aunque detectaron menos mutaciones, dieron mayor confianza en la permanencia del cierre y en la conservación del saldo ante eventos prohibidos. El número de fallos detectados depende de los defectos seleccionados y no permite declarar una técnica universalmente superior.

## Comparación de cobertura

| Técnica      | Líneas ejecutadas | Ramas ejecutadas | Mutaciones detectadas |
|--------------|-------------------|------------------|-----------------------|
| Equivalencia | 113/150           | 33/52            | 2/5                   |
| Límites      | 94/150            | 23/52            | 4/5                   |
| Decisiones   | 120/150           | 32/52            | 5/5                   |
| Estados      | 103/150           | 24/52            | 2/5                   |

Las mediciones proceden de ejecuciones separadas. Existe solapamiento: cero pertenece a una partición inválida y también marca una frontera; una cuenta congelada participa tanto en decisiones como en transiciones. Ese solapamiento aporta comprobaciones complementarias, pero impide sumar porcentajes. La unión cubre todo el

código instrumentado sin cubrir todas las secuencias ni todas las combinaciones semánticas. Persisten huecos como límites específicos de Premium, cambio de año, concurrencia y liquidación externa.

## Aplicación profesional

En un proyecto real se priorizarían decisiones financieras y límites monetarios por su impacto sobre fondos del cliente. Después se integrarían transiciones para bloqueos, recuperación y cierre, manteniendo particiones como base de validación de entradas. La combinación de decisiones y límites resulta especialmente eficaz: una determina cuándo corresponde una acción y la otra precisa dónde cambia el resultado. Los casos deberían ejecutarse también contra una API y un almacenamiento reales, con trazabilidad a requisitos aprobados.

## Recomendaciones

La siguiente iteración debería resolver ambigüedades con el responsable del producto antes de implementar, ampliar el catálogo de mutaciones y medir cobertura de requisitos y transiciones además de código. Conviene agregar pruebas de propiedades para conservación de fondos y generación de secuencias, junto con pruebas de integración, seguridad y carga. La lección central es que una cobertura alta resulta útil para localizar omisiones estructurales, mientras que la calidad del oráculo y la diversidad de escenarios determinan qué defectos puede detectar la suite.